SAP HANA Cloud, SAP HANA Database Administration Guide

Generated on: 2026-08-01 19:21:28 GMT+0000

SAP HANA Cloud, SAP HANA Database | QRC 2/2026

Public

Original content: https://help.sap.com/docs/HANA_CLOUD_DATABASE/f9c5015e72e04fffa14d7d4f7267d897?locale=en-US&state=PRODUCTION&version=2026_2_QRC

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit https://help.sap.com/docs/disclaimer.

Temporal Tables

Temporal tables are tables that provide functionality to manage historical data: that is, to maintain a set of current values that are valid for a predefined period and also maintain an auditable continuous history of changes.

System-Versioned Tables maintain a history of previous versions of each record based on validity periods and support time-based queries. In system-versioned tables the data is split across two tables: a current data table and a corresponding history table. They support the tracking of changes on column store tables by capturing the validity period of each record, it is then possible to read ranges of historical data and to query in 'time-travel' mode using the AS OF + <time-stamp> syntax.

Many applications, however, need to be able to manage and manipulate historical business data independent of system time-stamps, for example, to define the validity period of a stock unit or the duration of a customer contract. **Application-Time Period Tables** support this so that records in the database can also have an application-specific validity period and applications can perform time-based queries on business data with optimized access paths.

System-Versioned Tables

System-versioned tables are part of the SQL standard. They support the tracking of changes on column store tables by capturing the validity period of each record.

Specifically in SAP HANA, each record has a **Valid from** and **Valid to** value; these values are maintained by the system so that the mechanism is invisible to the user.

System-versioned tables always consist of two physical tables:

1. The main table of records that are currently valid.
2. A corresponding history table (a one-to-one correspondence) of archived records. A naming convention is not required but may be helpful, for example, to append "_history" to the name of the main table.

This section gives an introduction to system-versioned tables with examples and a commentary on the different types.

Full details of the syntax definition of system-versioned tables are given in the *SAP HANA SQL and System Views Reference* (see, for example, CREATE TABLE <system_versioning_spec>).

System-Versioned Tables (Main Table)

When you execute DML commands (insert, delete, update) on a system-versioned table, any records that are deleted or updated (and whose **Valid from** and **Valid to** columns are populated) are archived in the history table. Internal triggers are executed to manage the archiving process.

Options are available to determine at which point of a transaction the timestamp value is generated. SQL keywords START, END, and COMMIT are used for this purpose. The timestamp value is generated one of two ways:

- At the start of processing the current transaction.
- Using the COMMIT option, the timestamp relates to the time when the transaction is finally committed. This is referred to as a system-versioned table with transactional system-time.

History Tables

The history table has essentially the same structure as the main table but does not have the primary key constraint of the current table. The history table must be consistent with the main table (see examples below). If you convert an existing table to a history table, you must ensure that all requirements for consistency are met.

Only delete operations on history tables are allowed. This is permitted in order to support housekeeping. Inserts and updates are not allowed and generate a *feature-not-supported* error. As a workaround to this, it would be possible to temporarily drop the system versioning on the table and history table and reapply it later.

Archived records in the history table can still be accessed by referring to their timestamp values. To query a history table, you must provide one or more timestamp values with one of the following time operators:

- To select data that was valid at a single point in time, you can specify a timestamp with the 'timetravel' operator `AS OF`.
- To define time ranges, you can use the `FROM - TO` syntax or `BETWEEN - AND`.

This example gives basic syntax using `AS OF`:

```
SELECT * FROM <system_versioned_table> FOR SYSTEM_TIME AS OF '<utc-timestamp>'
```

A timetravel period can also be defined for the current session using a session variable so that the AS OF is no longer necessary with each command:

```
SET [SESSION] 'TEMPORAL_SYSTEM_TIME_AS_OF' = '2018-01-01';
SELECT * FROM My_sys_time_table;
```

Creating a Standard System-Versioned Table

You can either create new system-versioned tables (using `CREATE`), or you can modify an existing table with the `ALTER` statement so that it becomes a system-versioned table. The key syntax elements are shown here using `CREATE`.

The definition of the history table and the main table are shown here. Note that the history table ('my_table_history' in this example) must exist before it can be named in the main table definition.

For the history table, the date range columns are defined without a generation clause:

```
CREATE COLUMN TABLE my_table_history
(…
<data columns>
<validfrom_columnname>TIMESTAMP NOT NULL,
<validto_columnname>TIMESTAMP NOT NULL
)
```

The main table refers to the existing history table:

```
CREATE COLUMN TABLE my_table
(…
<data columns>
<validfrom_columnname> TIMESTAMP NOT NULL GENERATED ALWAYS AS ROW START,
<validto_columnname> TIMESTAMP NOT NULL GENERATED ALWAYS AS ROW END,
PERIOD FOR SYSTEM_TIME (validfrom_columnname, validto_columnname)
)
WITH SYSTEM VERSIONING HISTORY TABLE my_table_history [[NOT]VALIDATED];
```

- The date range columns are timestamps and are maintained by the system; they have a special generation type: *generated always as row start* and *generated always as row end*. The *Valid to* value of the current record is always set to

MAX_TIMESTAMP. These columns are then identified as *period for system_time* values. In this case the timestamp value is always generated at the start of processing the current transaction.

- The `WITH` clause specifies the corresponding history table for the archived data.
- An optional validation keyword is also supported.

Creating a System-Versioned Table with Transactional System-Time

The timestamp columns in this case are based on transactional system-time. The generation type for the **Valid from** column includes the optional COMMIT keyword; the effect of this is to generate the value for the timestamp when the transaction is finally committed.

As in the previous case, the history table must exist when the main table is created. If the COMMIT option is used, then the **Valid to** column in the history table must be defined with GENERATED ALWAYS AS ROW COMMIT:

```
CREATE COLUMN TABLE my_table_history
(…
<data columns>
<validfrom_columnname>TIMESTAMP NOT NULL,
<validto_columnname>TIMESTAMP NOT NULL GENERATED ALWAYS AS ROW COMMIT
)
```

The main table refers to the existing history table:

```
CREATE COLUMN TABLE my_table
(…
<data columns>
<validfrom_columnname> TIMESTAMP NOT NULL GENERATED ALWAYS AS ROW START COMMIT,
<validto_columnname> TIMESTAMP NOT NULL GENERATED ALWAYS AS ROW END,
PERIOD FOR SYSTEM_TIME (validfrom_columnname, validto_columnname)
)
WITH SYSTEM VERSIONING HISTORY TABLE my_table_history [[NOT]VALIDATED];
```

Triggers, including those used for referential constraints, are supported on system-versioned tables with transactional system-time.

Deleting Data

To permanently delete data from a system-versioned table, it is necessary to delete data from both main and history tables in the following sequence:

1. Submit a delete statement referencing the main table. This will delete the data in the main table and move the deleted versions into the history table.
2. Submit a delete statement referencing the history table. This will delete the data permanently.

System Views

Temporal tables can be identified in the following system views:

- In the `SYS.TABLES` view, `TEMPORAL_TYPE` is used to identify temporal tables. Possible values are: *Temporal* for the current table of a system-versioned table or an application-time period table, *History* or *NULL* for other table types.
- Details of system-versioned tables and their corresponding history table are available in the view `SYS.TEMPORAL_TABLES` (TABLE_NAME and HISTORY_TABLE_NAME). Temporal tables are further categorized in

SYS.TEMPORAL_TABLES.PERIOD_NAME where the value is either `SYSTEM_TIME` for the current table of a system-versioned table or `APPLICATION_TIME` for application-time period tables.

- In the `SYS.TABLE_COLUMNS` view, `GENERATION_TYPE` indicates either *ALWAYS AS ROW START* or *ALWAYS AS ROW END*.

Related Information

CREATE TABLE Statement (Data Definition)
ALTER TABLE Statement (Data Definition)

Application-Time Period Tables

Application-time period tables allow you to manage and manipulate historical business data based on application-specific time periods, which are independent of system timestamps.

Application-time period tables capture the time in which a record is valid in the business world rather than on the basis of the time it was entered in the database. These application-time periods can reflect the past, present, or future as determined by business needs and can be updated and managed from within the application. These tables are available only in the column store.

Some applications may require a history based on both system timestamps and business-oriented timestamps. For these scenarios, application-time period tables can be combined with system-versioned tables to form bitemporal tables.

Creating and Updating Application-Time Period Tables

Full details of the SQL commands available for working with application-time period tables are given in the *SAP HANA SQL Reference Guide*. See, for example, CREATE TABLE *<application_time_period_configuration>* and *<bi_temporal_table_spec>*.

Here we give a brief introduction to the essential commands with examples.

You can create an application-time period table with either CREATE or ALTER table commands using the clause PERIOD FOR APPLICATION_TIME (valid_from, valid_to).

To select records that have been valid at a certain point in time ('timetravel'), use the AS OF operator:

```
select * from <app_time_table> for application_time as of ‘<timestamp>’
```

A timetravel period can also be defined for the current session so that the AS OF is no longer necessary with each command:

```
SET [SESSION] 'TEMPORAL_APPLICATION_TIME_AS_OF' = '2018-01-01';
SELECT * FROM My_app_time_table;
```

You can update historical records within a specified time frame using the UPDATE FOR PORTION OF syntax. The following example sets the value of `customer_email` for the period January 2018:

```
UPDATE customers FOR PORTION OF application_time FROM '2018-01-01' TO '2018-01-31'
SET customer_email='new_email@acmecompany.de' WHERE customer_id=1;
```

This updates records where the ValidFrom and ValidTo dates overlap with the given portion dates. These records will be updated, and the validity periods will be adjusted as follows: the valid to date of the existing record is closed, a new record is inserted for the month of January, and a new record is inserted from the end of January:

CUSTOMER_ID	CUSTOMER_NAME	CUSTOMER_EMAIL	VALIDFROM	VALIDTO
1	ACME Company	info@acmecompany.de	2017-10-01	2018-01-01
1	ACME Company	new_email@acmecompany.de	2018-01-01	2018-01-31
1	ACME Company	info@acmecompany.de	2018-01-31	9999-12-31

System Views

The difference between the current table of a system-versioned table and an application-time period table can be seen in the database in SYS.TEMPORAL_TABLES.PERIOD_NAME. The value is either `SYSTEM_TIME` for the current table of a system-versioned table or `APPLICATION_TIME` for application-time period tables. Bitemporal tables have two entries in this view for both SYSTEM_TIME and APPLICATION_TIME.

The column SYS.TABLES.TEMPORAL_TYPE shows the value `TEMPORAL` if the table is an application-time period table or `HISTORY` for history tables of a system-versioned table.

Related Information

CREATE TABLE Statement (Data Definition)

Working with Temporal Tables

Transactional Tables Extend the Functions of History Tables

SAP HANA already supports history tables.

- Allow time-based queries on previous versions of a record.
- Write-operations don't physically overwrite an existing record, but insert a new version.
- Each version has a validity period, which is based on commit IDs.
- Proprietary implementation. Doesn't match SQL standard.

Challenges

The temporal information of a record ...

- ... is not externally accessible, e.g., by SQL.
- ... is specific to the database instance.
- ... has to be mapped to an accessible time-format, i.e., a TIMESTAMP.

For a long time, SAP HANA SQL has supported the creation of history tables. This was done by issuing a CREATE HISTORY COLUMN TABLE <table_name> SQL statement. As a result of this statement, a log of all transactions executed against a history table would be permanently retained in system tables. The log contained the unique commit ID of the transaction (one is created for each and every commit SQL statement executed in SQL) along with the UTC timestamp of when that commit executed. The commit ID and/or the UTC timestamp allowed a developer, when selecting records from a table, to retrieve not just the current version of the record but any and

all prior versions of that record that existed at any point in the past (known as a time travel query).

To retrieve prior versions of a record, the developer would execute the SET HISTORY SESSION TO <when> statement where <when> would be set to 1 for the unique commit ID, 2 for the UTC timestamp of when the commit occurred, or 3 using the SQL keyword NOW (indicating to only retrieve current versions of the record).

While useful, there were several limitations to using the history table approach. The first was that the commit ID was HANA-instance-specific and could not be migrated to other HANA instances. In addition, the commit ID was not externally accessible (from ABAP, for example). Finally, the design of history tables was not based off of any SQL standard.

SQL:2011 or ISO/IEC 9075:2011 was officially adopted in December 2011. Among other areas, it established a standardized way of designing temporal tables in database systems. SAP HANA 2.0 SPS03 and later supports the creation of temporal tables in accordance with the SQL:2011 specification.

A temporal table gives the developer not only the ability to execute "time travel" queries, but also to define validity periods for a record.

Introducing Temporal Tables

Temporal Tables are tables whose records are associated with one or more temporal periods.

System-Versioned Tables allow change tracking on database tables.

- Validity periods are automatically maintained by the database whenever

a record is changed.
- Retrospective changes to the validity period of a record are strictly prohibited. A new version is inserted for every change to preserve a records history.
- Temporal information must have a very high granularity.

Application-Time Period Tables capture the time, in which a record is valid in the real world.

- Validity periods are determined by the application.
- Applications can update validity period of a record, e.g., to correct errors.
- Temporal information can arbitrarily reflect the past, present or future at any granularity.

Bitemporal Tables combine system-versioned tables and application-time period tables.

Temporal periods are based on already existing SQL data types.

- Two columns, defining START and END.
- Temporal data type, e.g., TIMESTAMP.



When creating temporal tables, the developer has several choices.

1. The first option is to create system-versioned tables, which will result in any and all changes to records in the table being stored in an associated history table (not to be confused with history tables discussed previously). With system-versioned tables, specific validity period columns will be added to the table and the type of those columns will be the built-in HANA SQL type TIMESTAMP. With system-versioned tables,

any and all changes to a record result in a new version (and thus the start of a new validity period) of the record being created. Under no circumstances with system-versioned tables can the validity period of a record be retroactively adjusted. Therefore, every version of the record that has ever existed will be preserved with a corresponding start and end validity period.

2. The second option is to create application-time period tables. With this option, the developer will create their own validity period columns (with whatever names and SQL data types preferred) for a record that is freely modifiable based on business logic needs of record maintenance. Therefore, the start and end validity periods are adjustable based on business needs. In addition, the developer can issue transactions that change certain fields of a record without necessarily changing the validity period of the record. These validity period columns will be ignored for all system-versioned table operations.

3. The final option that the developer has in regard to temporal table definition is to apply both approaches. This is known as a bitemporal table definition. In this case, both system-versioned validity period fields and application-time period validity period fields will be created, allowing the developer to implement both change tracking and business logic validity in the table design.

SAP HANA Cloud, SAP HANA Database Modeling Guide for SAP Business Application Studio

Generated on: 2026-08-01 19:37:23 GMT+0000

SAP HANA Cloud, SAP HANA Database | QRC 2/2026

Public

Original content: https://help.sap.com/docs/HANA_CLOUD_DATABASE/d625b46ef0b445abb2c2fd9ba008c265?locale=en-US&state=PRODUCTION&version=2026_2_QRC

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit https://help.sap.com/docs/disclaimer.

Temporal Joins

A temporal join lets you set time intervals when to fetch records. A temporal join maps the time interval between the master data and the transaction data.

Temporal joins are based on the date and time field from the fact table, and the time interval (to and from fields) from the master data view. The date field from the fact table is called the temporal column.

If the temporal column values in the fact table are within the valid time interval values of their join partners from the master data view that means the respective records are selected. A time interval is assigned to each record in the results set. This means that the fact table can contain multiple versions of the same entry with different timestamps, and the version that is selected depends on the time interval defined in the master data view. The valid time range assigned to each record is determined by the temporal join definition, based on the columns specified as **Left Columns** and **Right Columns**.

The supported data types for **Temporal Column**, **From Column**, and **To Column** are timestamp, date, time, and integers only. For alternative ways to define temporal join logic, see Add System Time and Application Time: Temporal Tables and Create Non Equi Joins.

Temporal Conditions

Temporal condition values, which you define in temporal joins, help determine whether to include or exclude boundary values.

When the join condition is executed, the boundary values of the time interval are defined by the FROM and TO date fields of the master data view.

An overview of temporal conditions:

Temporal Condition	Meaning
Include To Exclude From	This temporal condition includes the value of the **To Column** field and excludes the value of the **From Column** field while executing the join.
Exclude To Include From	This temporal condition excludes the value of the **To Column** field and includes the value of the **From Column** field while executing the join.
Exclude Both	This temporal condition excludes the value from both the **To Column** field and the **From Column** field while executing the join.
Include Both	This temporal condition includes the value from both the **To Column** field and **From Column** field while executing the join.

Example: Temporal Joins

Create temporal joins to join the master data with the transaction data (fact table), based on a time column value from the transaction data and the time validity columns from the master data.

Consider a dimension calculation view named `PRODUCT` (master data) with attributes `PRODUCT_ID`, `VALID_FROM_DATE`, and `VALID_TO_DATE` and a calculation view of type cube, `SALES` (transactional data) with attributes `PRODUCT_ID`, `DATE`, and `REVENUE`.

Now, you can create a temporal join between the master data view and transaction data using the attribute `PRODUCT_ID` to analyze sales of the product for a particular period.